

Implementando Uma Lista Sequencial Com Vetor de Capacidade Fixa

(APP) Atividade Prática com Prazo de Entrega e Com Consulta – Peso 1

Objetivos Pedagógicos da Atividade

1. Aplicar o conceito de Tipo Abstrato de Dados (TAD) para listas lineares genéricas, identificando o papel da interface ***IListaLinear*** como contrato das operações de consulta, operações básicas de mutação e operações de iteração;
2. Relacionar a especificação abstrata do TAD com uma implementação concreta baseada em vetor estático, por meio da classe ***ListaSequencial***, percebendo vantagens e limitações da alocação sequencial;
3. Desenvolver a habilidade de manipular diretamente o ***array*** interno de uma estrutura de dados, implementando corretamente operações de inserção, atualização, remoção e limpeza, preservando: a ordem relativa dos elementos, o controle adequado do tamanho lógico da lista e a capacidade fixa do vetor;
4. Desenvolver a capacidade de projetar operações derivadas a partir de operações básicas, implementando métodos como ***inserirInicio***, ***removerInicio***, ***removerFim*** e ***removerElemento*** reutilizando métodos já existentes, reforçando princípios de reutilização de código e coesão;
5. Consolidar o entendimento da semântica de operações de mutação de uma lista linear, diferenciando claramente: inserção em posição específica, atualização de um elemento, remoção por posição, remoção por valor (primeira ocorrência) e esvaziamento total da estrutura;
6. Estimular o raciocínio sobre custos operacionais de listas sequenciais, percebendo o impacto de deslocamentos de elementos em operações de inserção e remoção em posições intermediárias da lista.

Implementações

1. Implemente o Tipo Abstrato de Dados *IListaLinear.java* (anexo a atividade) no *package estruturas.Lineares* - valor (0,5) pontos.
2. Implemente a classe concreta *ListaSequencial.java* (anexo a atividade) no *package estruturas.Lineares.estaticas.Lista* - valor (0,5) pontos.
3. Na classe *ListaSequencial.java*, complete o método com a seguinte assinatura:

public void inserir(T elemento, int posicao)

Este método deve inserir um elemento em uma posição específica da lista sequencial, manter a ordem relativa dos elementos e atualizar o tamanho da lista. Observações: Valide se a posição está dentro do intervalo permitido; Verifique se há capacidade disponível no vetor interno; Use deslocamento manual (*Loop*) ou *System.arraycopy()* para mover os elementos - valor (2,0) pontos.

4. Na classe *ListaSequencial.java*, complete o método com a seguinte assinatura:

public void atualizar(int posicao, T elemento)

Este método deve atualizar (substituir) o elemento armazenado em uma posição específica da lista sequencial, manter inalterados os demais elementos e o tamanho da lista. Observações: Valide se a posição está dentro do intervalo válido; Atualização deve ser feita por substituição direta no *array* interno - valor (1,5) ponto.

5. Na classe *ListaSequencial.java*, complete o método com a seguinte assinatura:

public T remover(int posicao)

Este método deve remover e retornar o elemento de uma posição específica da lista sequencial, atualizar o tamanho da lista e manter a ordem relativa dos elementos restantes. Observações: Valide se a posição está dentro do intervalo válido; Use deslocamento manual (*Loop*) ou *System.arraycopy()* para mover os elementos - valor (2,0) pontos.

6. Na classe *ListaSequencial.java*, complete o método com a seguinte assinatura:

public void limpar()

Este método deve remover todos os elementos da lista sequencial, restaurando-a ao estado inicial, redefinir o tamanho da lista e manter a capacidade original do vetor interno. Observações: A lista deve estar pronta para reutilização imediatamente após a limpeza; Use *Arrays.fill()* ou um *Loop* para definir como *null* todas as posições ocupadas - valor (0,5) pontos.

7. Na classe *ListaSequencial.java*, complete o método com a seguinte assinatura:

public void inserirInicio(T elemento)

Este método deve inserir um elemento no início da lista sequencial. Observações: Este método é uma operação derivada que deve reutilizar alguma operação básica - valor (0,5) pontos.

8. Na classe *ListaSequencial.java*, complete o método com a seguinte assinatura:

public void removerElemento(T elemento)

Este método deve remover a primeira ocorrência de um elemento específico da lista sequencial, se existir. Deve atualizar o tamanho da lista. Observações: Este método é uma operação derivada que deve reutilizar alguma operação básica; A remoção deve ser feita apenas na primeira ocorrência do elemento - valor (1,5) ponto.

9. Na classe *ListaSequencial.java*, complete o método com a seguinte assinatura:

public void removerInicio()

Este método deve remover o elemento do início da lista sequencial. Observações: Não deve retornar o elemento removido. O Método é uma operação derivada que deve reutilizar alguma operação básica - valor (0,5) pontos.

10. Na classe *ListaSequencial.java*, complete o método com a seguinte assinatura:

public void removerFim()

Este método deve remover o elemento do final da lista sequencial. Observações:
Não deve retornar o elemento removido. O método é uma operação derivada que deve reutilizar alguma operação básica - valor (0,5) pontos.

Resultado Esperado

Ao executar o **método main** da classe **AppListaSequencial** (anexa a atividade) o resultado deve ser parecido com a imagem abaixo:

```
1. Lista criada com capacidade 5
   Estado: []
   Tamanho: 0
   Vazia: true

2. Inserindo elementos com inserir(elemento, posicao):
   Inserir 10 na posição 0: [10]
   Inserir 30 na posição 1: [10, 30]
   Inserir 20 na posição 1: [10, 20, 30]
   Tamanho atual: 3

3. Testando atualizar(posicao, elemento):
   Atualizar posição 0 para 100: [100, 20, 30]
   Atualizar posição 2 para 300: [100, 20, 300]

4. Testando remover(posicao):
   Remover posição 1: [100, 300]
   Elemento removido: 20
   Tamanho atual: 2

5. Testando inserirInicio(elemento):
   Inserir 5 no início: [5, 100, 300]
   Inserir 1 no início: [1, 5, 100, 300]
   Tamanho atual: 4

6. Testando removerElemento(elemento):
   Remover elemento 100: [1, 5, 300]
   Tentar remover elemento 999 (não existe): [1, 5, 300]

7. Testando removerInicio():
   Remover início: [5, 300]
   Remover início novamente: [300]

8. Testando removerFim():
   Lista após novas inserções: [40, 50, 60, 300]
   Remover fim: [40, 50, 60]
   Remover fim novamente: [40, 50]

9. Testando limpar():
   Lista após limpar(): []
   Tamanho: 0
   Vazia: true

10. Testando reutilização após limpeza:
    Inserir 1000 no início e 2000 na posição 1: [1000, 2000]

11. Testando casos especiais:
    Inserir null na posição 0: [null, 1000, 2000]
    Remover elemento null: [1000, 2000]
    Estado final da lista: [1000, 2000]
    Tamanho final: 2
    Está vazia: false
```